Assignment-10.4 2303A51738

Task 1: AI-Assisted Syntax and Code Quality Review

Python script (with errors):

10.4(1).py > ...

```python
def calculate_average(numbers)
total = 0
    for num in number:
        total += num
    avg = total / len(numbers)
    return average

def printResult(value):
print("The result is: " + value)

nums = [10, 20, 30, 40, 50]

result = calculateAverage(nums)

printresult(result
)
```

10.4(1).py | Untitled-2

10.4(1).py > ...

```python
def calculate_average(numbers):
    total = 0
    for num in numbers:
        total += num
    avg = total / len(numbers)
    return avg

def printResult(value):
    print("The result is: " + str(value))

nums = [10, 20, 30, 40, 50]

result = calculate_average(nums)

printResult(result)
```

PROBLEMS OUTPUT TERMINAL Python

```
PS C:\Users\gbsgu\OneDrive\Documents\Desktop\Assisted coding> & C:/Users/gbsgu/AppD
ata/Local/Programs/Python/Python312/python.exe "c:/Users/gbsgu/OneDrive/Documents/D
esktop/Assisted coding/10.4(1).py"
The result is: 30.0
PS C:\Users\gbsgu\OneDrive\Documents\Desktop\Assisted coding>
```

CHAT

ERROR IDENTIFICATION AND CORRECTION EXPLANATION

identify all syntactic and structural errors ,correct them systematically and Generate an explanation of each fix made

Used 1 reference

Reviewed and updated Untitled-1

Explanation of Fixes Made

I identified and corrected several syntactic and structural errors in the code. Here's a systematic breakdown of each fix:

1. **Missing colon in function definition**: Added `:` after `def calculate_average(numbers)` to properly define the function. In Python, function definitions require a colon to indicate the start of the function body.
2. **Incorrect indentation**: Fixed the indentation for all lines inside the `calculate_average` function. Python uses indentation to define code blocks, so `total = 0`, the `for` loop, and subsequent lines needed to be indented consistently (typically 4 spaces).
3. **Typo in loop variable**: Changed `for num in number:` to `for num in numbers:`. The parameter is named `numbers`, not `number`, so the loop should iterate over the correct variable.
4. **Incorrect return variable**: Changed `return average` to `return avg`. The calculated average is stored in the variable `avg`, not `average`.
5. **Missing colon in second function definition**: Added `:` after `def printResult(value)` to properly define the function.
6. **Incorrect indentation in second function**: Fixed the indentation for the `print` statement inside `printResult`. It needed to be indented to be part of the function body.
7. **Type error in string concatenation**: Changed `"The result is: " + value` to `"The result is: " + str(value)`. Since `value` is a number (float), it must be converted to a string before concatenation with another string.
8. **Incorrect function name in call**: Changed `result = calculateAverage(nums)` to `result = calculate_average(nums)`. Function names are case-sensitive in Python, and the defined function is `calculate_average`, not `calculateAverage`.
9. **Incorrect function name and missing parenthesis**: Changed `printresult(result` to `printResult(result)`. The function name should match the definition (`printResult`), and added the missing closing parenthesis `)` to complete the function call.

The code now runs without syntax errors and produces the correct output: calculating the average of the numbers and printing it.

10.4(1).py

Describe what to build next

Local Agent Auto

Task 2: Performance-Oriented Code Review.

```python
def find_duplicates(numbers):
    duplicates = []
    for i in range(len(numbers)):
        for j in range(i + 1, len(numbers)):
            if numbers[i] == numbers[j] and numbers[i] not in duplicates:
                duplicates.append(numbers[i])
    return duplicates


data = [1, 2, 3, 4, 5, 2, 3, 6, 7, 8, 1, 9]

result = find_duplicates(data)
print("Duplicates:", result)
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS
PS C:\Users\gbsgu\OneDrive\Documents\Desktop\Assisted coding> ^C
PS C:\Users\gbsgu\OneDrive\Documents\Desktop\Assisted coding> & C:/Users/gbsgu/AppData/Local/Programs/Python/Python312/python.exe "c:/Users/gbsgu/OneDrive/Documents/Desktop/Assisted coding/10.4(2).py"
Duplicates: [1, 2, 3]
PS C:\Users\gbsgu\OneDrive\Documents\Desktop\Assisted coding>
```

Ai refactored the code :

```python
10.4(2).py > ...
def find_duplicates(numbers):
    seen = set()
    duplicates = set()
    for num in numbers:
        if num in seen:
            duplicates.add(num)
        else:
            seen.add(num)
    return list(duplicates)


data = [1, 2, 3, 4, 5, 2, 3, 6, 7, 8, 1, 9]

result = find_duplicates(data)
print("Duplicates:", result)
```

Explanation:

CHAT

← ERROR IDENTIFICATION AND CORRECTION EXPLANATION

This function works, but it seems slow with large inputs.
Can you analyze it for performance problems, explain what makes it inefficient, and rewrite it in a more optimized way without changing the result?
Please also explain how your changes improve performance.

> Used 1 reference

✓ Reviewed 10.4(2).py and updated it

Performance Analysis and Optimization

Original Implementation Issues

The original `find_duplicates` function has significant performance problems:

1. **Time Complexity:** $O(n^2)$ - The nested loops result in comparing each element with every subsequent element. For a list of size n, this performs approximately n(n-1)/2 comparisons.
2. **Inefficient Membership Check:** The condition `numbers[i] not in duplicates` requires scanning the `duplicates` list (which grows during execution), making each check O(k) where k is the current number of duplicates found. This compounds the inefficiency.
3. **Redundant Comparisons:** Even after finding a duplicate, the algorithm continues checking the same value against remaining elements.

For large inputs (e.g., n=10,000), this could require ~50 million operations, making it impractically slow.

Optimized Implementation

I rewrote the function using sets for O(1) average-case lookups:

```python
def find_duplicates(numbers):
    seen = set()
    duplicates = set()
    for num in numbers:
        if num in seen:
            duplicates.add(num)
        else:
            seen.add(num)
    return list(duplicates)
```

Performance Improvements

1. **Time Complexity:** Reduced to O(n) - Single pass through the list with constant-time set operations.
2. **Space Complexity:** O(n) in worst case (when all elements are unique), but this is acceptable compared to the $O(n^2)$ time cost of the original.
3. **Eliminated Redundancy:** Each element is processed exactly once, and duplicates are tracked efficiently without repeated comparisons.
4. **Set Operations:** Python sets provide average O(1) lookup and insertion, making the algorithm much faster for large datasets.

Verification

The optimized version produces identical results (unique duplicate values) but scales linearly with input size. For the example data `[1, 2, 3, 4, 5, 2, 3, 6, 7, 8, 1, 9]`, it correctly returns `[1, 2, 3]`.

Grok Code Fast 1 • 1x

Task 3: Readability and Maintainability Refactoring.

10.4(3).py > sum_even_numbers

```python
def sum_even_numbers(numbers):
    """
    Calculate the sum of all even numbers in the given list.
    This function iterates through a list of integers and sums up
    only the even numbers (numbers divisible by 2). Odd numbers
    are ignored in the summation.
    Args:
        numbers (list of int): A list of integers to process.
    Returns:
        int: The sum of all even numbers in the list.
    Examples:
        >>> sum_even_numbers([1, 2, 3, 4, 5, 6])
        12
        >>> sum_even_numbers([1, 3, 5])
        0
    """
    total = 0
    for num in numbers:
        if num % 2 == 0:
            total += num
    return total


numbers_list = [1, 2, 3, 4, 5, 6, 7, 8]
print(sum_even_numbers(numbers_list))
```

Explanation:

ERROR IDENTIFICATION AND CORRECTION EXPLANATION

so, Refator the code and make sure to apply PEP 8 formatting standards ,Improve naming conventions Add meaningful Documentation And Explain dockstrings fucntion purpose

> Used 1 reference

✓ Reviewed and updated 10.4(3).py

Refactoring Summary

I've refactored the code to follow PEP 8 standards, improved naming conventions, and added comprehensive documentation. Here are the key changes:

PEP 8 Compliance

- **Function naming:** Changed `f` to `sum_even_numbers` (snake_case convention)
- **Spacing:** Added spaces around operators (`=`, `+`, `%`, `==`)
- **Indentation:** Ensured consistent 4-space indentation
- **Line length:** Kept lines appropriately short
- **Redundant code:** Removed the unnecessary `else: r = r` block

Improved Naming Conventions

- `f` → `sum_even_numbers` : Descriptive function name indicating its purpose
- `x` → `numbers` : Clear parameter name showing it expects a list of numbers
- `r` → `total` : Meaningful variable name for the accumulator
- `i` → `num` : Better loop variable name (though `i` was acceptable, `num` is more descriptive)
- `d` → `numbers_list` : Descriptive name for the data list

Documentation and Docstrings

Added a comprehensive docstring that explains the function's purpose:

- **Description:** Clearly states what the function does (sums even numbers)
- **Args:** Documents the parameter type and description
- **Returns:** Specifies the return type and what it represents
- **Examples:** Provides doctest-style examples showing usage and expected output

The docstring serves as both documentation and a specification of the function's behavior, making the code self-documenting and easier to understand for other developers. The function now clearly communicates its intent through both its name and documentation.

Task 4: Secure Coding and Reliability Review

```python
import sqlite3
def get_user(username):
    """
    Retrieve user information from the database by username.
    This function safely queries the users table using parameterized queries
    to prevent SQL injection attacks.
    Args:
        username (str): The username to search for.
    Returns:
        list: A list of tuples containing user data, or empty list if not found.
    Raises:
        ValueError: If username is empty or invalid.
        sqlite3.Error: If there's a database error.
    """
    if not username or not isinstance(username, str):
        raise ValueError("Username must be a non-empty string")
    try:
        with sqlite3.connect("users.db") as conn:
            cursor = conn.cursor()
            # Use parameterized query to prevent SQL injection
            query = "SELECT * FROM users WHERE username = ?"
            cursor.execute(query, (username,))
            result = cursor.fetchall()
            return result
    except sqlite3.Error as e:
        # Log the error in production
        print(f"Database error: {e}")
        raise
# Example usage with input validation
try:
    user_input = input("Enter username: ").strip()
    if not user_input:
        print("Username cannot be empty")
    else:
        result = get_user(user_input)
        if result:
            print("User found:", result)
        else:
            print("User not found")
except ValueError as e:
    print(f"Input error: {e}")
except sqlite3.Error as e:
    print(f"Database error: {e}")
```

Explanation:

ERROR IDENTIFICATION AND CORRECTION EXPLANATION

review this Python code for security and reliability issues.Identify any security vulnerabilities, refactor the code using safe coding practices, add proper exception handling, and improve its overall robustness for production use.
Also briefly explain the improvements you made.

> Used 1 reference

✓ Reviewed 10.4(4).py and updated 10.4(4).py

Security and Reliability Review & Improvements

Identified Issues in Original Code

Security Vulnerabilities:

- **SQL Injection:** The query was constructed using string concatenation (`"SELECT * FROM users WHERE username = '" + username + "'"`), allowing attackers to inject malicious SQL code through the `username` parameter.

Reliability Issues:

- **No Exception Handling:** Database connection failures, query errors, or file access issues would cause unhandled exceptions and crashes.
- **Resource Leaks:** Database connections weren't properly closed if exceptions occurred (no context manager).
- **No Input Validation:** Empty or invalid usernames could cause unexpected behavior.
- **Poor Error Messages:** Users would see raw database errors, potentially exposing sensitive information.

Refactored Code Improvements

1. **SQL Injection Prevention:**
   - Replaced string concatenation with parameterized queries (`cursor.execute(query, (username,))`).
   - This ensures user input is treated as literal data, not executable SQL.
2. **Proper Resource Management:**
   - Used `with sqlite3.connect("users.db") as conn:` context manager to automatically close connections even if exceptions occur.
3. **Exception Handling:**
   - Added try-except blocks to catch `sqlite3.Error` and `ValueError` .
   - Provides user-friendly error messages without exposing internal details.
4. **Input Validation:**
   - Added checks for empty or non-string usernames.
   - Raises `ValueError` for invalid inputs before database operations.
5. **Documentation:**
   - Added comprehensive docstring explaining function purpose, parameters, return values, and exceptions.
6. **Improved User Experience:**
   - Added `.strip()` to user input to handle accidental whitespace.
   - Provides clear feedback for different scenarios (user found/not found, errors).
7. **Production Readiness:**
   - Error messages suggest logging for production use.
   - Code is now robust against common failure modes.

The refactored code maintains the same functionality while being secure against SQL injection and resilient to common runtime errors. It's now suitable for production use with proper logging and monitoring.

Task 5: AI-Based Automated Code Review Report

Code:

10.4(5).py > ...

```python
def calc(a,b):
    try:
        c=a/b
        print("res:",c)
    except:
        print("error")

def process(d):
    total=0
    for i in d:
        total=total+i
    print("t:",total)

def check(n):
 if n>100:
  print("big")
 else:
  print("small")

x=10
y=0
calc(x,y)

data=[1,2,3,4,5]
process(data)

n=150
check(n)

for i in range(3):
 print("loop",i)

try:
    z = int("abc")
except:
    pass
```

Review Report:

Analyze the code and prepare structured code review report that evaluates
code redability,Naming conventions,formatting and style consistency ,error handling ,documenation quality ,maintainability

Used 1 reference

Read 10.4(5).py

Structured Code Review Report

1. Code Readability

Rating: Poor

Analysis: The code lacks clarity and is difficult to understand at a glance. Function names like `calc`, `process`, and `check` provide no insight into their purpose. Variable names (`a`, `b`, `c`, `d`, `i`, `n`, `x`, `y`, `z`) are cryptic and don't convey meaning. There's no logical flow or grouping of related operations.

Issues:

- No comments explaining logic
- Mixed concerns (calculation, processing, checking) without clear separation
- Print statements scattered throughout make output hard to follow

Recommendations:

- Use descriptive function and variable names (e.g., `calculate_division`, `numbers_list`, `total_sum`)
- Add inline comments for complex logic
- Group related functions and separate business logic from I/O

2. Naming Conventions

Rating: Poor

Analysis: The code violates Python naming conventions (PEP 8). Function names use camelCase instead of snake_case. Variables are single letters, making them meaningless.

Issues:

- Functions: `calc` → should be `calculate_division`
- Variables: `a`, `b` → should be `dividend`, `divisor`
- No type hints for parameters or return values

Recommendations:

- Adopt snake_case for functions and variables
- Use descriptive names that explain purpose
- Add type hints (e.g., `def calculate_division(dividend: float, divisor: float) -> float:`)

3. Formatting and Style Consistency

Rating: Poor

Analysis: Inconsistent formatting throughout. Indentation varies, spacing around operators is missing, and line breaks are arbitrary.

Issues:

- Inconsistent indentation (mix of spaces, possible tabs)
- No spaces around operators: `c=a/b` , `total=total+i`
- Cramped code: multiple statements on one line, no blank lines between functions
- Inconsistent quoting (no strings shown, but generally inconsistent)

Recommendations:

- Use 4 spaces for indentation consistently
- Add spaces around operators and after commas
- Follow PEP 8 line length limits (79 characters)
- Add blank lines between functions and logical sections

4. Error Handling

Rating: Poor

Analysis: Exception handling is inadequate and potentially dangerous. Bare `except:` clauses catch all exceptions, masking real issues.

Issues:

- `except:` in `calc` catches everything, including `ZeroDivisionError`, but doesn't handle it properly
- Last `try-except` with `pass` silently ignores errors
- No specific exception types handled
- No logging or meaningful error messages

Recommendations:

- Catch specific exceptions (e.g., `ZeroDivisionError`, `ValueError`)
- Provide meaningful error messages or logging
- Avoid bare `except:` clauses
- Consider raising exceptions instead of just printing

5. Documentation Quality

Rating: Poor

Analysis: Completely lacking. No docstrings, module docstring, or inline documentation.

Issues:

- No function docstrings explaining purpose, parameters, or return values
- No module-level documentation
- No comments explaining complex logic

ERROR IDENTIFICATION AND CORRECTION EXPLANATION

- Use 4 spaces for indentation consistently
- Add spaces around operators and after commas
- Follow PEP 8 line length limits (79 characters)
- Add blank lines between functions and logical sections

4. Error Handling

Rating: Poor

Analysis: Exception handling is inadequate and potentially dangerous. Bare `except:` clauses catch all exceptions, masking real issues.

Issues:

- `except:` in `calc` catches everything, including `ZeroDivisionError`, but doesn't handle it properly
- Last `try-except` with `pass` silently ignores errors
- No specific exception types handled
- No logging or meaningful error messages

Recommendations:

- Catch specific exceptions (e.g., `ZeroDivisionError`, `ValueError`)
- Provide meaningful error messages or logging
- Avoid bare `except:` clauses
- Consider raising exceptions instead of just printing

5. Documentation Quality

Rating: Poor

Analysis: Completely lacking. No docstrings, module docstring, or inline documentation.

Issues:

- No function docstrings explaining purpose, parameters, or return values
- No module-level documentation
- No comments explaining complex logic

Recommendations:

- Add module docstring at the top
- Include docstrings for all functions using Google/NumPy style
- Add inline comments for non-obvious code
- Consider README or usage examples

6. Maintainability

Rating: Poor

Analysis: The code is hard to modify, test, or extend. Tight coupling between functions and global state makes changes risky.

Issues:

- Global variables mixed with functions

- Global variables mixed with functions
- No separation of concerns (I/O, calculation, validation all mixed)
- Hard-coded values (e.g., `100` in `check`)
- No unit tests possible due to print statements and global state
- Difficult to reuse functions independently

Recommendations:

- Separate I/O from business logic
- Make functions pure (no side effects like printing)
- Use constants for magic numbers
- Add unit tests
- Break into smaller, focused functions

Overall Assessment

Grade: F (Needs Complete Rewrite)

This code demonstrates basic Python syntax but fails all modern software development standards. It would be unacceptable in any professional or collaborative environment. A complete refactoring following PEP 8, with proper naming, error handling, and documentation, is strongly recommended before any production use or further development.